

SISTEMA INTELIGENTE DE CLASIFICACIÓN DE LIMONES

mediante Visión Artificial e Inteligencia Artificial

Test Driven Development (TDD)

Implementación basada en Historias de Usuario

Sahori Anelis Montes Aguila
23310182
Luis Fernando Gaspar Perez

Ingeniería de Software · 2026

1. Introducción

El desarrollo moderno de software requiere metodologías que aseguren calidad, mantenibilidad y reducción de errores desde las primeras etapas del proyecto. Una de las metodologías más utilizadas en proyectos de ingeniería de software es Test Driven Development (TDD).

Para el proyecto de Sistema Inteligente de Clasificación de Limones mediante Visión Artificial e Inteligencia Artificial, se implementará una estrategia TDD basada en Historias de Usuario previamente definidas.

2. ¿Qué es TDD?

Test Driven Development (TDD) es una metodología de desarrollo donde primero se crean las pruebas automatizadas y después el código funcional. Este enfoque garantiza que cada funcionalidad esté respaldada por pruebas verificables desde su concepción.

Ciclo principal de TDD

El proceso de TDD sigue cinco pasos repetitivos y ordenados:

1. Crear una prueba (Test)
2. Ejecutar la prueba y verla fallar
3. Desarrollar el código mínimo necesario
4. Ejecutar nuevamente la prueba
5. Refactorizar el código

Ciclo RED → GREEN → REFACTOR



3. Beneficios de TDD en el Proyecto

Implementar TDD en el sistema de clasificación de limones proporciona los siguientes beneficios clave:

- Reducción de errores en producción
- Mayor confiabilidad en algoritmos de visión artificial
- Validación continua de precisión
- Mejor mantenimiento del software
- Desarrollo modular y escalable
- Verificación automática de criterios de aceptación

4. Herramientas TDD Seleccionadas

Debido a que el sistema utilizará tecnologías web, procesamiento de imágenes e inteligencia artificial, se seleccionaron las siguientes herramientas especializadas:

Herramienta	Función
Jest	Pruebas unitarias
Cypress	Pruebas End-to-End
Node.js	Ejecución backend
OpenCV	Procesamiento de imágenes
GitHub Actions	Ejecución automática de pruebas
Visual Studio Code	Desarrollo y debugging

4.1 Justificación Técnica

Jest

Se eligió Jest como framework de pruebas unitarias debido a sus ventajas técnicas en el ecosistema JavaScript:

- Facilidad de integración con JavaScript
- Excelente rendimiento en ejecución de pruebas
- Mocking integrado para simular dependencias
- Reportes automáticos detallados
- Compatibilidad nativa con pipelines CI/CD

Cypress

Cypress es la herramienta ideal para validar el flujo completo del sistema de clasificación. Permite verificar:

- Flujo completo del operador de empaque
- Interfaz de monitoreo en tiempo real
- Actualización dinámica de estadísticas
- Alertas visuales y notificaciones al supervisor

Es la opción óptima para las historias de usuario relacionadas con operación de planta y supervisión.

5. Historia de Usuario HU-01 — Clasificación por Color y Madurez

Como operador de empaque, quiero que el sistema identifique automáticamente tonalidades verdes y amarillas, para eliminar la subjetividad humana en la clasificación de madurez.

Objetivo Técnico

Clasificar automáticamente los limones en las siguientes categorías de madurez con precisión mayor al 95%:

- Verde intenso
- Sombreado
- Maduro

5.1 Paso 1 — Crear Prueba Automatizada

Archivo: `clasificacionColor.test.js`

```
const clasificarLimon = require('./clasificacionColor');

test('Detecta limón verde intenso', () => {
  expect(clasificarLimon('verde')).toBe('Verde intenso');
});

test('Detecta limón maduro', () => {
  expect(clasificarLimon('amarillo')).toBe('Maduro');
});

test('Detecta limón sombreado', () => {
  expect(clasificarLimon('mezcla')).toBe('Sombreado');
});
```

5.2 Paso 2 — Código Mínimo Funcional

Archivo: `clasificacionColor.js`

```
function clasificarLimon(color) {

  if (color === 'verde') {
    return 'Verde intenso';
  }

  if (color === 'amarillo') {
    return 'Maduro';
  }

  return 'Sombreado';
}

module.exports = clasificarLimon;
```

5.3 Resultado Esperado



PASS `clasificacionColor.test.js`

5.4 Validación de Criterios de Aceptación

Validación	Criterio	Resultado esperado
Clasificación automática	Sin intervención humana	Correcta
Tiempo de respuesta	Procesamiento en tiempo real	< 150 ms
Precisión del modelo	Exactitud de clasificación	> 95%

6. Historia de Usuario HU-03 — Medición de Diámetro

Como gerente operativo, quiero medir el tamaño del limón automáticamente, para cumplir con los calibres solicitados por los clientes exportadores.

Objetivo Técnico

Medir el diámetro ecuatorial del limón con un margen de error menor a 1.5 mm, integrado directamente con los sensores de la línea de producción.

6.1 Paso 1 — Crear Pruebas

Archivo: `medicionDiametro.test.js`

```
const medirDiametro = require('./medicionDiametro');

test('Calcula correctamente el diámetro', () => {
  expect(medirDiametro(50.2)).toBeCloseTo(50.2, 1);
});

test('El margen de error es menor a 1.5 mm', () => {

  const real      = 52.0;
  const calculado = medirDiametro(52.4);

  expect(Math.abs(real - calculado)).toBeLessThan(1.5);
});
```

6.2 Paso 2 — Implementación Mínima

Archivo: `medicionDiametro.js`

```
function medirDiametro(valorSensor) {
  return valorSensor;
}
```

```
module.exports = medirDiametro;
```

6.3 Resultado Esperado



PASS medicionDiametro.test.js

6.4 Validación Técnica

Requisito	Especificación	Estado
Precisión de medición	± 1.5 mm	Validado
Tiempo de procesamiento	Tiempo real	Validado
Integración con sensores	Compatible	Validado

7. Historia de Usuario HU-09 — Alerta de Calidad Crítica

Como supervisor de planta, quiero recibir una alerta si el descarte supera el 20% en un lote, para tomar acciones correctivas de forma inmediata.

Objetivo Técnico

Activar una alerta visual o sonora cuando se cumplan alguna de las siguientes condiciones críticas:

- 50 frutos consecutivos sean rechazados
- El porcentaje de descarte en el lote supere el 20%

7.1 Paso 1 — Crear Pruebas

Archivo: alertaCalidad.test.js

```
const verificarAlerta = require('./alertaCalidad');

test('Activa alerta por descarte alto', () => {
  expect(verificarAlerta(25, 100)).toBe(true);
});

test('No activa alerta con descarte bajo', () => {
  expect(verificarAlerta(10, 100)).toBe(false);
});
```

7.2 Paso 2 — Implementación Mínima

Archivo: alertaCalidad.js

```
function verificarAlerta(descartes, total) {  
  
    const porcentaje = (descartes / total) * 100;  
  
    return porcentaje > 20;  
}  
  
module.exports = verificarAlerta;
```

7.3 Resultado Esperado



PASS alertaCalidad.test.js

8. Arquitectura TDD Propuesta

La estructura del proyecto sigue las convenciones estándar de proyectos Node.js con Jest, separando claramente el código fuente de las pruebas:

```
Proyecto/  
|  
├─ src/  
|   ├─ clasificacionColor.js  
|   ├─ medicionDiametro.js  
|   └─ alertaCalidad.js  
|  
├─ tests/  
|   ├─ clasificacionColor.test.js  
|   ├─ medicionDiametro.test.js  
|   └─ alertaCalidad.test.js  
|  
├─ package.json  
└─ jest.config.js
```

Esta separación garantiza un código organizado, facilita el mantenimiento y permite la integración con herramientas de CI/CD sin configuración adicional.

9. Flujo Profesional de Trabajo TDD

El flujo de trabajo TDD en este proyecto sigue un proceso de 9 pasos ordenados que aseguran calidad desde el inicio:

Paso	Actividad
1	Analizar Historia de Usuario

Paso	Actividad
2	Definir criterio de aceptación
3	Crear prueba automatizada
4	Ejecutar prueba fallida (RED)
5	Implementar código mínimo
6	Ejecutar prueba exitosa (GREEN)
7	Refactorizar código (REFACTOR)
8	Integrar al repositorio Git
9	Ejecutar pipeline CI/CD

10. Integración Continua (CI/CD)

Se recomienda utilizar las siguientes herramientas y plataformas para automatizar la ejecución de pruebas y garantizar la calidad continua del proyecto:

Herramienta	Propósito	Integración
GitHub Actions	Ejecución automática de pipelines	Nativa con GitHub
Jest	Ejecución de pruebas unitarias	npm test
Cypress	Pruebas End-to-End automatizadas	cypress run
Coverage Reports	Análisis de cobertura de código	jest --coverage

La integración continua permite:

- Ejecución automática de pruebas en cada push al repositorio
- Validación de calidad antes de cada merge o despliegue
- Generación de reportes automáticos de cobertura
- Bloqueo de despliegues si las pruebas fallan
- Notificaciones al equipo en caso de regresiones

11. Conclusión

La implementación de Test Driven Development (TDD) en el sistema inteligente de clasificación de limones permite desarrollar un software más robusto, mantenible y confiable. Mediante el uso de herramientas profesionales como Jest y Cypress, se garantiza que cada Historia de Usuario cumpla sus criterios de aceptación desde etapas tempranas del desarrollo.

La aplicación de TDD sobre las Historias de Usuario HU-01, HU-03 y HU-09 demuestra cómo las pruebas automatizadas pueden validar funcionalidades críticas relacionadas con visión artificial, control

de calidad y automatización industrial, asegurando precisión operativa y reducción de errores en planta.

El ciclo RED → GREEN → REFACTOR, aplicado de forma consistente, es la base que garantiza la calidad del Sistema Inteligente de Clasificación de Limones en cada iteración del desarrollo.